

# Compositional Polymorphism: Resolving the Object Composition Trilemma

R. Scott McKinney  
*scott@manifold.systems*

## Abstract

We identify a fundamental trilemma in object composition: no prior model satisfies runtime polymorphism, independent late-bound components, and static typing together. We present *compositional polymorphism*, a composition model that satisfies all three properties. The model is realized as a compiler plugin for Java requiring no language changes, supports arbitrary DAG composition with per-interface dispatch roots, introduces composite-scoped encapsulation, and delivers dispatch performance equivalent to standard virtual dispatch.

## Contents

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                      | <b>2</b>  |
| <b>2</b> | <b>The Model</b>                         | <b>3</b>  |
| <b>3</b> | <b>Implementation</b>                    | <b>7</b>  |
| 3.1      | Plugin Architecture . . . . .            | 7         |
| 3.2      | Two-Phase Compilation . . . . .          | 7         |
| 3.3      | Component Transformation . . . . .       | 8         |
| 3.4      | Self-Call Rewriting . . . . .            | 8         |
| 3.5      | Composite Class Transformation . . . . . | 9         |
| 3.6      | Abstract Component Support . . . . .     | 9         |
| 3.7      | @internal Enforcement . . . . .          | 10        |
| <b>4</b> | <b>Evaluation</b>                        | <b>10</b> |
| 4.1      | Dispatch Performance . . . . .           | 10        |
| 4.2      | Expressiveness . . . . .                 | 10        |
| 4.3      | Test Coverage . . . . .                  | 11        |
| 4.4      | Limitations . . . . .                    | 11        |
| 4.5      | Safety Guarantees . . . . .              | 11        |
| <b>5</b> | <b>Related Work</b>                      | <b>12</b> |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>6</b> | <b>Discussion</b>                      | <b>14</b> |
| 6.1      | Language Design Implications . . . . . | 14        |
| 6.2      | Concurrency . . . . .                  | 14        |
| 6.3      | Future Work . . . . .                  | 14        |
| <b>7</b> | <b>Conclusion</b>                      | <b>14</b> |
| <b>A</b> | <b>Summary Table</b>                   | <b>16</b> |

# 1 Introduction

Object-oriented programming is built on polymorphism. We identify a fundamental trilemma in OO composition: every mainstream mechanism sacrifices at least one of three properties:

1. **Runtime polymorphism** — self-calls within a component dispatch through the composite host (the *Self problem* [1, 5]: a self-call in a component should invoke the composite’s override, not the component’s own implementation)
2. **Independent, late-bound components** — components are independent objects supplied at construction time, not statically inlined, fused, or ordered into a linear chain; a component may simultaneously carry a distinct composite identity per interface
3. **Static typing** — the composition’s interface contracts are enforced at compile time

Prototype-based delegation [1, 5] solves the Self problem and provides late-bound components, but without static type guarantees. Scala traits [6] and Rust trait impls enforce static typing and support polymorphism, but collapse to a single linearized type at link time—real components are gone. Python’s MRO and Ruby’s mixins linearize the chain, forfeiting independent component identity and, in the case of dynamic languages, static typing. Java interfaces with default methods provide static typing but no delegation semantics. Kotlin’s `by keyword` provides forwarding, not delegation—self-calls do not dispatch through the composite.

Even the closest prior models fall short. Kniesel’s type-safe delegation [8] addresses the Self problem under static typing but restricts each component to a single composite identity, precluding DAG composition. Ostermann’s delegation layers [9] achieve correct self-call dispatch and static typing, but components must be defined as virtual nested classes of a common enclosing family type—independently defined classes cannot be composed.

This paper presents **compositional polymorphism**, which satisfies all three properties — and does so at dispatch cost equivalent to standard virtual dispatch, making the model practical for production use. The core contributions are:

- A formal model of compositional polymorphism with ten definitions and one central proposition

- A compiler transformation for standard Java (8–26) implementing the model with O(1) dispatch
- Composite-scoped encapsulation (**@internal**) — component-family visibility without package-scope leakage; a principled alternative to **protected**
- Abstract components with deferred linkage (**asLink()** factory)
- A safety mechanism for cycles and premature access (**DelegationLinkageError**)
- An open-source implementation supporting Java 8–26 with full IDE integration (IntelliJ) and a test suite covering generics, DAG topology, diamond resolution, abstract components, default methods, and **@internal** enforcement

## 2 The Model

We define compositional polymorphism over a standard typed object model where classes implement interfaces and method dispatch is defined by the receiver’s runtime type.

**Definition 1** (Component). *A component is a class annotated **@part**. Components implement one or more interfaces. A component may contain **@link** fields (making it a composite).*

**Definition 2** (Link). *A link is a field annotated **@link** in a host class. A link field is **private** and **final**. The delegation surface of a link is the intersection of the field’s declared type and the interfaces of the enclosing class (or the subset specified by **value()**).*

**Definition 3** (Composite). *A composite is a class containing one or more **@link** fields. A composite implements the union of the delegation surfaces of all its links.*

**Definition 4** (Delegation surface). *The delegation surface of a composite is the set of interfaces delegated by at least one link. Each interface in the surface is assigned to exactly one link; ambiguity (diamond overlap) is a compile error unless resolved with **share=**.*

**Definition 5** (Composite identity). *When a component is wired to a composite, it receives a composite identity: a reference to the composite object. Self-calls within the component—calls to methods in the component’s interfaces—dispatch through the composite identity rather than **this**.*

**Definition 6** (Transitive composability). *A composite may itself be a component. When a composite-component is wired to a larger composite, composite identity propagates transitively: each interface in the composite-component is re-rooted at the new outermost composite.*

**Definition 7** (Delegation surface restriction). *An interface in a component’s **implements** clause may be annotated **@internal** to exclude it from the component’s delegation surface. An **@internal** interface participates in the component’s own behavior but cannot be delegated by external composites.*

**Definition 8** (Composite-scoped method). *An interface method annotated **@internal** is accessible within the composite family (component-to-component via link fields) but is not acces-*

sible through the composite’s external interface. It is the compositional analog of **protected** without package-scoping.

**Definition 9** (Abstract component). *A component may be declared **abstract**. Abstract methods in the component are obligations imposed on the delegating composite. The compiler generates a static **asLink()** factory per non-private constructor; **asLink()** returns a deferred instance whose abstract methods throw **DelegationLinkageError** until wired.*

**Definition 10** (Diamond resolution). *When two links in a composite share a common interface, exactly one must declare **share=T** for the shared interface. That link is authoritative; the other delegates only its non-shared interfaces. Unresolved diamonds are compile errors.*

**Proposition 1.** *For any component  $P$  in a delegation chain for interface  $I$  rooted at composite  $C$ : a self-call **this.m()** within  $P$  where  $m \in I$  dispatches to  $C$ . A formal proof in a small-step operational semantics is left to future work (Section 6.3).*

The guarantee holds because wiring is completed before the composite is observable: at any point a self-call can occur, the dispatch slot for  $I$  already holds the outermost composite in the chain, so dispatch is correct by construction.

## DAG composition example

The following example requires all three trilemma properties to produce the correct output and exposes the limits of every prior model. It illustrates Definition 6 (transitive composability): **WorkerPart** implements both **Logger** and **Processor** and, when composed into **App** via **LoggingWorker**, carries distinct composite identities for each interface (Figures 1 and 2).

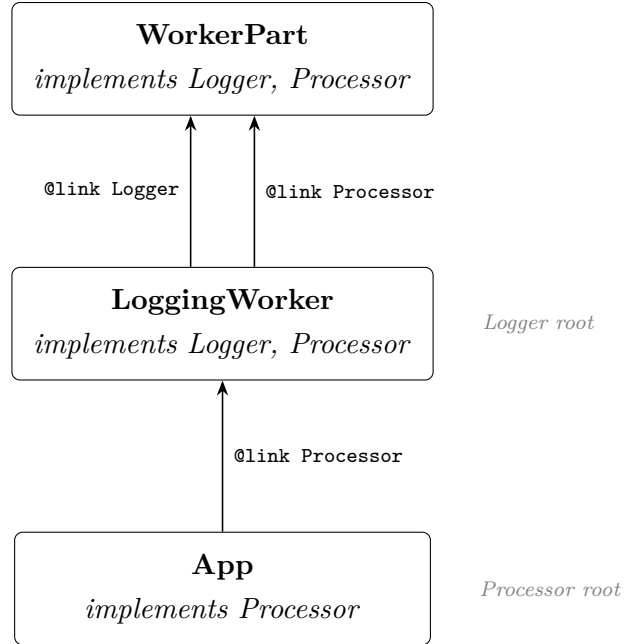


Figure 1: DAG composition structure. Arrows show `@link` fields labeled by the delegated interface. **LoggingWorker** is the root for **Logger** dispatch; **App** is the root for **Processor** dispatch.

```

interface Logger {
    String prefix();
    void log(String msg);
}

interface Processor {
    String label();
    String process(String s);
}

@part class WorkerPart implements Logger, Processor {
    public String prefix() { return "WORKER"; }
    public String label() { return "worker"; }
    public void log(String msg) { System.out.println "[" + prefix()
        + "]" + msg); }
    public String process(String s) {
        log("processing: " + s); // Logger dispatch
        return label() + ":" + s; // Processor dispatch
    }
}

@part class LoggingWorker implements Logger, Processor {
    @link Logger logger;
    @link Processor processor;
    LoggingWorker(WorkerPart w) { this.logger = w; this.processor =
        w; }
    @Override public String prefix() { return "LOGGED"; }
}

class App implements Processor {
    @link Processor worker = new LoggingWorker(new WorkerPart());
    @Override public String label() { return "APP"; }
}

new App().process("hello");
// Logger dispatch rooted at LoggingWorker -> prefix() ->
// "LOGGED"
// Processor dispatch rooted at App -> label() -> "APP"
// prints: "[LOGGED] processing: hello"
// returns: "APP:hello"

```

Figure 2: DAG composition with multiple composite roots per component. `WorkerPart` carries distinct composite identities for `Logger` (rooted at `LoggingWorker`) and `Processor` (rooted at `App`).

This example is not expressible with any prior model in Table 1. With Kotlin’s `by`, self-

calls in `WorkerPart` resolve within `WorkerPart` itself — `prefix()` returns `"WORKER"` and `label()` returns `"worker"` regardless of the composite host, because `by` is forwarding, not delegation. With Scala traits, the composition is fixed at compile time: `WorkerPart` cannot be supplied as a runtime object, and a single component cannot carry distinct composite identities for distinct interfaces. All three trilemma properties are required to produce the observed output.

## 3 Implementation

### 3.1 Plugin Architecture

The implementation is a `javac` plugin operating on the compiler's AST during two phases: `ENTER` (structural generation) and `ANALYZE` (semantic rewriting). No bytecode manipulation is performed; all transformation is at the source AST level and subject to standard `javac` type checking.

### 3.2 Two-Phase Compilation

**ENTER phase.** The plugin registers a `TaskListener` that fires on `ENTER` events. For each `@part` class, the plugin:

- injects the `$PartClass` marker interface (enables `O(1)` `instanceof` checks);
- generates the composite identity table field (`$selves`) as a compile-time-sized array indexed by interface ordinal;
- generates the `$linkPartToSelf` wiring method, which propagates composite identity through the DAG.

For abstract `@part` classes, the plugin additionally:

- generates the private `$Impl` concrete subclass (shell at `ENTER` start; body completed before `ANALYZE`);
- generates `asLink()` static factories (one per non-private constructor) before `ANALYZE`.

For each composite class (any class containing `@link` fields), the plugin:

- resolves the delegation surface of each `@link` field;
- generates method declaration skeletons for each surface method not already implemented by the composite (bodies completed at `ANALYZE` time).

**ANALYZE phase.** After type resolution, the plugin:

- rewrites self-calls in `@part` method bodies (`this.m(args) → $selves[IDX_I].m(args)`) where `IDX_I` is the compile-time ordinal for the interface declaring `m`;
- in composites: rewrites `@link` field assignments to invoke `$linkPartToSelf()` recursively on the assigned component;

- generates delegation method bodies for each skeleton: `return link.m(args)` for non-void methods, `link.m(args)` for void; the authoritative link for each method is the `@link` field assigned to interface *I*, accounting for `share=` declarations;
- completes `$Impl` inner classes: overrides all abstract methods with `DelegationLink-ageError` stubs;
- enforces `@internal` visibility at call sites.

### 3.3 Component Transformation

Each `@part` class is transformed to hold a `$selves` array indexed by interface ordinal. The ordinals are assigned at compile time—stable across compilations of the same source.

```
$selves: Object[] (length = number of delegated interfaces)
// initialized: $selves[i] = this (before wiring)
// after wiring: $selves[IDX_I] = composite identity for
//               interface I
```

Initializing to `this` means an unwired component dispatches on itself; once wired, dispatch routes through the composite identity. `$linkPartToSelf` updates each relevant `$selves` slot to the composite root, then recurses through any `@link` fields in the component to propagate identity transitively through the DAG. Cycle detection is performed at each slot assignment.

### 3.4 Self-Call Rewriting

Within a `@part` method body, every call `this.m(args)` where `m` is declared in interface *I* is rewritten to:

```
((I) $selves[IDX_I]).m(args)
```

The cast to *I* is required for `invokeinterface`. `IDX_I` is a compile-time integer literal.

More broadly, any use of `this` where the contextual type is a declared interface is rewritten to `$selves[IDX_I]`; uses where the contextual type is the concrete `@part` type are compile errors.

**Self-preservation.** Because the model guarantees that self-calls dispatch through the composite host, it must statically enforce the invariants that make that guarantee hold. A `@part` class may not reference `this` except as one of its declared interfaces: passing `this` as the concrete `@part` type or assigning it to a variable of the concrete type are compile errors. Field access via `this` is permitted — a part may call back through its own `@link` fields, which serves as the compositional analog of `super`.

```
@part class MyPart implements MyInterface {
  public void method() {
    passMyPart(this);           // compile error: this as MyPart
    MyPart p = this;           // compile error
```



```

    MyInterface i = this; // rewritten: $selves[IDX_MyInterface]
    passMyInterface(this); // rewritten: $selves[IDX_MyInterface]
  }
}

```

If **this** leaked as the concrete type, a caller could hold a direct reference to the component and invoke methods on it outside the composite dispatch path, violating Proposition 1. The restriction is the static price of the dispatch guarantee. Other composition models do not impose it because they do not make an equivalent guarantee.

**Default methods.** Default methods in the delegation surface receive generated override bodies that route dispatch through the composite identity when wired, and fall back to the interface default when not yet wired. Self-calls to default methods within **@part** method bodies are rewritten at the call site identically to other self-calls, requiring no transformation of the interface source.

### 3.5 Composite Class Transformation

In the composite class, each **@link** field assignment:

```
this.link = expr;
```

is rewritten to:

```
this.link = expr.$linkPartToSelf(this, linkScope);
```

`linkScope` identifies the delegation scope. The call is guarded by an `instanceof $PartClass` check (O(1) marker interface test) so that non-**@part** assignments pass through unchanged. Wiring occurs once at construction time.

### 3.6 Abstract Component Support

For an abstract **@part** class `Foo`, the compiler generates:

```

private static final class $Impl extends Foo {
  // For each abstract method m in Foo:
  @Override public T m(Params p) {
    throw new DelegationLinkageError(
      "abstract method called before linkage: Foo.m");
  }
}

public static Foo asLink() { return new $Impl(); }
// one factory per non-private constructor:
public static Foo asLink(ParamType p) { return new $Impl(p); }

```

`asLink()` results are valid `Foo` references and can be passed freely. If an abstract method is called before the instance is wired to a composite, `DelegationLinkageError` is thrown. Once wired, the abstract method stubs are unreachable because self-calls dispatch through `$selves[IDX_I]`, which resolves to the composite.

### 3.7 @internal Enforcement

For `@internal` interfaces in a `@part`'s `implements` clause:

- the interface is excluded from the component's delegation surface;
- composite classes are prevented from declaring a `@link` field of that interface type backed by the component;
- a compile error is issued at violation sites.

For `@internal` methods on interfaces:

- the method is excluded from the composite's external interface;
- calls to `composite.internalMethod()` from outside the composite family are compile errors;
- calls via a `@link` field reference (`this.link.internalMethod()`) from within the composite are permitted.

## 4 Evaluation

### 4.1 Dispatch Performance

The dispatch path for a self-call through a composite is:

1. Load `$selves[IDX_I]` — array indexed load, compile-time constant index
2. Cast to *I*
3. `invokeinterface m` — standard JVM virtual dispatch

This is functionally  $O(1)$ . The parallel to vtable dispatch is direct: a vtable call is an indexed slot load followed by an indirect call. In practice, JIT compilers inline trivial forwarders entirely. Composite DAGs are shallow and their call sites are monomorphic or bimorphic in the common case—the JIT's most aggressively inlined scenarios. Composite dispatch performance is therefore equivalent to direct virtual dispatch in practice.

### 4.2 Expressiveness

The model supports four composition patterns not achievable together in prior static OO models:

1. **Independent component identity** — components are real objects; their internal state is independent
2. **Polymorphic self-dispatch** — self-calls route through the outermost composite in the delegation chain for the dispatched interface
3. **Multiple composite roots per component** — a component wired for multiple interfaces simultaneously carries distinct composite identities per interface
4. **Composite-scoped encapsulation** — `@internal` provides finer-grained visibility than `public/protected/private` for component families

### 4.3 Test Coverage

The implementation is tested against a suite covering:

- forwarding vs. delegation semantics
- DAG composition at depth  $\geq 3$
- diamond detection and resolution
- abstract component / `asLink()` lifecycle
- `@internal` interface and method enforcement
- default method dispatch
- inheritance chains in `@part` hierarchies
- cycle detection (runtime `DelegationLinkageError`)

### 4.4 Limitations

- Using `@link`, `@part`, and `@internal` requires configuring the `javac` plugin in the build system (Maven, Gradle, or similar).
- Composite dispatch semantics are not enforced under Java reflection. This is by design.
- The plugin relies on internal `javac` APIs for AST rewriting. These APIs are not part of the official public compiler API and may change across JDK versions. This is a practical constraint of the `javac` plugin approach; it does not affect the model's correctness or generality.

### 4.5 Safety Guarantees

**Cycle detection.** If a component is wired to itself directly or transitively, `DelegationLinkageError` is thrown at assignment time during the recursive `$linkPartToSelf()` call.

**Deferred method access.** If an `asLink()`-produced instance has an abstract method called before wiring, `DelegationLinkageError` is thrown at the stub. This is the compositional analog of `AbstractMethodError`.

**@internal violations.** All `@internal` enforcement is at compile time; no runtime overhead.

**Self-preservation.** `@part` classes are not permitted to reference `this` in a context other than a declared interface. This statically prevents component identity from escaping the composite dispatch path, ensuring Proposition 1 holds for all reachable call sites. Field access via `this` is permitted.

## 5 Related Work

**Prototype-based delegation.** Lieberman [1] introduced the delegation concept: an object may delegate unanswered messages to a prototype, with *self* remaining the original receiver. Self [5] realized this in a working language. These systems solve the Self problem and support late-bound components but provide no static type guarantees and rely on hash-based message dispatch.

**Behavioral subtyping.** Liskov and Wing [2] established the substitutability principle underpinning interface contracts in statically-typed OO. Because self-calls in a component dispatch through the composite host, the composite satisfies the behavioral contracts of all its components—substitutability is preserved across every delegation boundary without special treatment.

**Mixins.** Bracha and Cook [3] formalized mixins as parameterized abstract subclasses. Mixins extend through linearization—the mixin is applied to a chain, not composed with independent objects. Self-calls dispatch within the linearized result, not through an independent component. No independent late-bound components result.

**Traits.** Schärli et al. [4] proposed traits as stateless, composable units of behavior. Trait composition is a compile-time operation: methods are flattened into the target class. There are no runtime component objects, no late binding, and no polymorphic dispatch through an independent component’s self-calls.

**Scala traits / abstract type members.** Odersky and Zenger [6] introduced statically-typed mixin composition with abstract type members. Scala’s `with` clause linearizes traits into the target class at compile time. Late binding is limited to the linearized type—independent components as runtime objects are not preserved.

**Multiple inheritance.** Stroustrup [7] introduced virtual base classes and virtual inheritance in C++ to handle diamonds. The result is a single linearized object with a merged vtable. Independent component identity is lost.

**Kniesel / Lava.** Kniesel’s “type-safe delegation” [8] (ECOOP 1999) addresses the Self problem under static typing using immutable delegation links in the Lava language.

Key convergences with compositional polymorphism: the Self problem is solved, delegation links are immutable (our `@link` fields are `private final`), and static type safety is enforced.

Key divergences: Lava’s delegation model is single-rooted—a component cannot carry independent composite identities for distinct interfaces simultaneously. Compositional polymorphism supports DAG composition where a single component does. Lava has no equivalent of `@internal`. Abstract components with deferred linkage (`asLink()`) have no direct Lava counterpart.

**Delegation Layers.** Ostermann’s delegation layers [9] (ECOOP 2002) are the closest prior model to compositional polymorphism. Self-calls within a role dispatch through the composed aggregate (P1), and type safety is enforced (P3) — but only within a family constraint: components must be defined as virtual nested classes of a common enclosing owner type. Independently defined classes from unrelated hierarchies cannot be composed, so P2 fails. Compositional polymorphism imposes no such structural constraint: any class implementing the target interface may serve as a component.

**Virtual classes and family polymorphism.** The virtual-class lineage—gbeta, CaesarJ, Object Teams/Java, and J& [11]—achieves open class composition through virtual nested classes within a common enclosing family type; Ernst, Ostermann, and Cook [10] provide the definitive formal treatment. Self-calls dispatch correctly within the family and static typing is enforced, satisfying P1 and P3. P2 fails across the entire lineage: components are class-level members of an enclosing family type, not independent objects wired at construction time. Introducing a new component combination requires defining a new family subtype—it is class-level inheritance, not runtime object assembly.

**Dynamic composition.** Newspeak [12] treats classes as first-class late-bound arguments, enabling runtime composition of independent module objects with correct self-call dispatch (P1, P2). Static type checking is absent by design (P3 fails). This is the strongest known example of a language that deliberately satisfies P1 and P2 at the cost of P3.

**Kotlin by-delegation.** Kotlin’s `by` keyword [13] generates forwarding methods at compile time. The delegated object’s self-calls resolve within the delegated object—the composite host is invisible. This is forwarding, not delegation. The Self problem is not addressed.

## 6 Discussion

### 6.1 Language Design Implications

The model does not require language changes to Java. All three properties—runtime polymorphism, independent late-bound components, and static typing—are achieved through AST rewriting within the existing type system.

This suggests that the trilemma is not fundamental to statically-typed OO languages per se; it is a consequence of how composition is currently specified and implemented in those languages. Extending any statically-typed OO language with a composition model equivalent to `@link/@part` would suffice.

The Java specifics that make the plugin approach tractable:

- `javac` plugins have full AST read/write access during compilation;
- `invokeinterface` in the JVM supports interface-typed dispatch independently of the concrete class hierarchy;
- interfaces as first-class types allow the `$selves` array to be typed at the interface level.

### 6.2 Concurrency

The DAG structure is favorable for concurrent use. Component wiring occurs during construction, before the object is shared. Once wired, `$selves` is effectively immutable—no synchronization is required for dispatch. Components may contain mutable state, subject to the usual Java concurrency rules; the dispatch mechanism adds no additional synchronization points.

### 6.3 Future Work

**Intersection type syntax.** The natural expression of multi-interface linking—`@link Foo & Bar fooBar;`—would eliminate the `value()` workaround. This requires syntactic extension to the Java parser within the `javac` plugin; feasibility is under investigation.

**Formal verification.** A formal soundness proof for the dispatch model (Proposition 1) in a small-step operational semantics would strengthen the theoretical foundation. The model maps cleanly to a subset of Featherweight Java; this is planned as follow-on work.

**Role-oriented programming.** The `@internal` mechanism partially overlaps with role-oriented programming models. A formal comparison is left to future work.

## 7 Conclusion

We have presented compositional polymorphism, a model that satisfies all three properties of the trilemma — runtime polymorphism, independent late-bound components, and static

typing — realized as a compiler plugin for Java. The implementation operates at compile time via AST rewriting, delivers dispatch performance equivalent to standard virtual dispatch, and supports DAG composition to arbitrary depth, composite-scoped encapsulation, and abstract components with deferred linkage. The trilemma is not fundamental to statically-typed OO languages; it is a consequence of how composition has been modeled. Any statically-typed OO language extended with an equivalent composition model would satisfy all three properties.

## References

- [1] H. Lieberman. Using prototypical objects to implement shared behavior in object-oriented systems. In *Proceedings of OOPSLA*, pages 214–223, 1986.
- [2] B. Liskov and J. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.
- [3] G. Bracha and W. Cook. Mixin-based inheritance. In *Proceedings of OOPSLA/ECOOP*, pages 303–311, 1990.
- [4] N. Schärli, S. Ducasse, O. Nierstrasz, and A. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP*, LNCS 2743, pages 248–274, 2003.
- [5] D. Ungar and R. B. Smith. Self: The power of simplicity. In *Proceedings of OOPSLA*, pages 227–242, 1987.
- [6] M. Odersky and M. Zenger. Scalable component abstractions. In *Proceedings of OOPSLA*, pages 41–57, 2005.
- [7] B. Stroustrup. Multiple inheritance for C++. In *Proceedings of the EUUG Spring Conference*, 1987.
- [8] G. Kniesel. Type-safe delegation for run-time component adaptation. In *Proceedings of ECOOP*, LNCS 1628, pages 351–366, 1999. DOI: 10.1007/3-540-48743-3\_16.
- [9] K. Ostermann. Dynamically composable collaborations with delegation layers. In *Proceedings of ECOOP*, LNCS 2374, pages 89–110, 2002.
- [10] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *Conference Record of POPL*, pages 270–282, 2006.
- [11] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *Proceedings of OOPSLA*, pages 347–366, 2006.
- [12] G. Bracha, P. von der Ahé, V. Bykov, Y. Kashai, W. Maddox, and E. Miranda. Modules as objects in Newspeak. In *Proceedings of ECOOP*, LNCS 6183, pages 405–428, 2010.
- [13] JetBrains. Delegation — Kotlin programming language. <https://kotlinlang.org/docs/delegation.html>, 2024.
- [14] R. S. McKinney. manifold-delegation. <https://github.com/manifold-systems/manifold/tree/master/manifold-deps-parent/manifold-delegation>, 2026.

## A Summary Table

| Model                                  | Self<br>solved | Independent<br>late-bound components | Static<br>typing |
|----------------------------------------|----------------|--------------------------------------|------------------|
| Prototype delegation [1, 5]            | ✓              | ✓                                    | ×                |
| Mixins [3]                             | ×              | ×                                    | ✓                |
| Traits [4]                             | ×              | ×                                    | ✓                |
| Kotlin <code>by</code> [13]            | ×              | ✓                                    | ✓                |
| Kniesel/Lava [8]                       | ✓              | ×                                    | ✓                |
| Delegation Layers [9]                  | ✓              | ×                                    | ✓                |
| Virtual classes [10, 11]               | ✓              | ×                                    | ✓                |
| Newspeak [12]                          | ✓              | ✓                                    | ×                |
| <b>Compositional polymorphism [14]</b> | ✓              | ✓                                    | ✓                |

Table 1: Comparison of composition models across the three trilemma properties.